System-wide Profiling & Tracing



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



System-wide Profiling & Tracing

- Sometimes, the problems are not tied to an application but rather due to the usage of multiple layers (drivers, application, kernel).
- In that case, it might be useful to analyze the whole stack.
- The kernel already includes a large number of tracepoints that can be recorded using specific tools.
- New tracepoints can also be created statically or dynamically using various mechanisms (kprobes for instance).



kprobes



Kprobes

- Allows to insert breaks at almost any kernel address dynamically and to extract debugging and performance information
- Uses code patching to modify text code to insert calls to specific handlers
  - `kprobes` allows to execute specific handlers when the hooked instruction is executed
  - `kretprobes` will trigger when returning from a function allowing to extract the return value of functions but also display the parameters that were used for the function call
- Needs some basic kernel configuration:
  - CONFIG_KPROBES=y to enable general kprobe support
  - CONFIG_KALLSYMS_ALL=y to allow hooking probes using `<symbol_name>` instead of raw function address
  - CONFIG_KPROBE_EVENTS=y to enable kprobes usage as tracing events in `tracefs`
- At the lowest level, k(ret)probes are manipulated with dedicated kernel APIs, allowing to write our own kprobe tools (eg as kernel modules)
- Can also be used from userspace with `/sys/kernel/tracing/kprobe_events`
- See trace/kprobes for more information



Basic kprobe tracing (1/2)

- Add a kprobe on `do_sys_openat2`:

```
$ echo "p:my_probe do_sys_openat2" > /sys/kernel/tracing/kprobe_events
```

- Add a kprobe in the same function but at a specific offset, and capture some arguments

```
$ echo "p:my_probe_2 do_sys_openat2+0x7c file=%r2" > /sys/kernel/tracing/kprobe_events
```

- Insert a kretprobe

```
$ echo 'r:my_retprobe do_sys_openat2 $retval' > /sys/kernel/tracing/kprobe_events
```



Basic kprobe tracing (2/2)

- Show existing kprobes

```
$ cat /sys/kernel/tracing/kprobe_events
```

- Enable a kprobe (ie: start capturing the corresponding event)

```
$ echo 1 > /sys/kernel/tracing/events/kprobes/my_probe/enable
```

- Get data emitted by kprobes

```
$ cat /sys/kernel/tracing/trace
```

- Delete a kprobe

```
$ echo "-:my_probe" >> /sys/kernel/tracing/kprobe_events
```



perf



perf

- *perf* allows to do a wide range of tracing and recording operations.
- The kernel already contains events and tracepoints that can be used. The list is given using `perf list`.
  - Syscall tracepoints must be explicitly enabled in kernel configuration using CONFIG_FTRACE_SYSCALLS.
- New probes can be created dynamically on any symbols and registers when debug info are not present.
- Tracing functions, recording variables and parameters content using their names will require a kernel compiled with CONFIG_DEBUG_INFO.
- If perf does not find `vmlinux` you have to provide it using `-k <vmlinux>`.



perf example

- List all events that matches `syscalls:*`

```
$ perf list syscalls:*
List of pre-defined events (to be used in -e):

  syscalls:sys_enter_accept                          [Tracepoint event]
  syscalls:sys_enter_accept4                         [Tracepoint event]
  syscalls:sys_enter_access                          [Tracepoint event]
  syscalls:sys_enter_adjtimex_time32                 [Tracepoint event]
  syscalls:sys_enter_bind                            [Tracepoint event]
...
```

- Record all `syscalls:sys_enter_read` events for `sha256sum` command into `perf.data` file.

```
$ perf record -e syscalls:sys_enter_read sha256sum /bin/busybox
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.018 MB perf.data (215 samples) ]
```



perf report example

- Display the collected samples ordered by time spent.

```
$ perf report
Samples: 591  of event 'cycles', Event count (approx.): 393877062
Overhead  Command      Shared Object                     Symbol
  22,88%  firefox-esr  [nvidia]                          [k] _nv031568rm
   3,21%  firefox-esr  ld-linux-x86-64.so.2              [.] __minimal_realloc
   2,00%  firefox-esr  libc.so.6                         [.] __stpncpy_ssse3
   1,86%  firefox-esr  libglib-2.0.so.0.7400.0           [.] g_hash_table_lookup
   1,62%  firefox-esr  ld-linux-x86-64.so.2              [.] _dl_strtoul
   1,56%  firefox-esr  [kernel.kallsyms]                 [k] clear_page_rep
   1,52%  firefox-esr  libc.so.6                         [.] __strncpy_sse2_unaligned
   1,37%  firefox-esr  ld-linux-x86-64.so.2              [.] strncmp
   1,30%  firefox-esr  firefox-esr                       [.] malloc
   1,27%  firefox-esr  libc.so.6                         [.] __GI___strcasecmp_l_ssse3
   1,23%  firefox-esr  [nvidia]                          [k] _nv013165rm
   1,09%  firefox-esr  [nvidia]                          [k] _nv007298rm
   1,03%  firefox-esr  [kernel.kallsyms]                 [k] unmap_page_range
   0,91%  firefox-esr  ld-linux-x86-64.so.2              [.] __minimal_free
```



perf probe

- *perf* allows to create dynamic tracepoints on both kernel functions and user-space functions.
- In order to be able to insert probes, CONFIG_KPROBES must be enabled in the kernel.
  - Note: *libelf* is required to compile *perf* with *probe* command support.
- New dynamic probes can be created and then used using *perf record*.
- Often on embedded platforms, `vmlinux` is not present on the target and thus only symbols and registers can be used.



perf probe examples (1/3)

- List all the kernel symbols that can be probed (no debug info needed):

```
$ perf probe --funcs
```

- Create a new probe on `do_sys_openat2` with *filename* named parameter (debug info required).

```
$ perf probe --vmlinux=vmlinux_file do_sys_openat2 filename:string
Added new event:
  probe:do_sys_openat2 (on do_sys_openat2 with filename:string)
```

- Execute `tail` and capture previously created probe event:

```
$ perf record -e probe:do_sys_openat2 tail /var/log/messages
...
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.003 MB perf.data (19 samples) ]
```



perf probe examples (2/3)

- Display the recorded tracepoints with *perf script*:

```
$ perf script
tail   164 [000]  3552.956573: probe:do_sys_openat2: (c02c3750) filename_string="/etc/ld.so.cache"
tail   164 [000]  3552.956642: probe:do_sys_openat2: (c02c3750) filename_string="/lib/tls/v7l/neon/vfp/libresolv.so.2"
...
```

- Create a new probe to capture the return value from `ksys_read`

```
$ perf probe ksys_read%return \$retval
```

- Execute `sha256sum` and capture previously created probe events:

```
$ perf record -e probe:ksys_read__return sha256sum /etc/fstab
```



perf probe examples (3/3)

- List all probes that have been created:

```
$ perf probe -l
  probe:ksys_read__return (on ksys_read%return with ret)
```

- Remove an existing tracepoint:

```
$ perf probe -d probe:ksys_read__return
```



perf record example

- Record all events for all cpus (system-wide mode):

```
$ perf record -a
^C
```

- Display recorded events from perf.data using `perf script`

```
$ perf script
...
klogd    85 [000]   208.609712:     116584   cycles:          b6dd551c memset+0x2c (/lib/libc.so.6)
klogd    85 [000]   208.609898:     121267   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34 (vmlinux)
klogd    85 [000]   208.610094:     127434   cycles:          c02f3ef4 kmem_cache_alloc+0xd0 (vmlinux)
 perf   130 [000]   208.610311:     132915   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34 (vmlinux)
 perf   130 [000]   208.619831:     143834   cycles:          c0a44cf4 _raw_spin_unlock_irqrestore+0x3c
(vmlinux)
klogd    85 [000]   208.620048:     143834   cycles:          c01a07f8 syslog_print+0x170 (vmlinux)
klogd    85 [000]   208.620241:     126328   cycles:          c0100184 vector_swi+0x44 (vmlinux)
klogd    85 [000]   208.620434:     128451   cycles:          c096f228 unix_dgram_sendmsg+0x46c (vmlinux)
kworker/0:2-mm_    44 [000]   208.620653:     133104   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34
(vmlinux)
 perf   130 [000]   208.620859:     138065   cycles:          c0198460 lock_acquire+0x184 (vmlinux)
...
```



Using perf trace

- `perf trace` captures and displays all tracepoints/events that have been triggered when executing a command

```
$ perf trace -e "net:*" ping -c 1 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
      0.000 ping/37820 net:net_dev_queue(skbaddr: 0xffff97bbc6a17900, len: 98,
        name: "enp34s0")
      0.005 ping/37820 net:net_dev_start_xmit(name: "enp34s0",
        skbaddr: 0xffff97bbc6a17900, protocol: 2048, len: 98,
        network_offset: 14, transport_offset_valid: 1, transport_offset: 34)
      0.009 ping/37820 net:net_dev_xmit(skbaddr: 0xffff97bbc6a17900, len: 98,
        name: "enp34s0")
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.867 ms
```



Using perf top

- `perf top` allows to do a live analysis of the running kernel
- It will sample all function calls and display them ordered by most time consuming one.
- This allows to profile the whole system usage

```
$ perf top Samples: 19K of event 'cycles', 4000 Hz, Event count (approx.): 4571734204 lost: 0/0 drop: 0/0
Overhead  Shared Object                          Symbol
   2,01%  [nvidia]                               [k] _nv023368rm
   0,94%  [kernel]                               [k] __static_call_text_end
   0,89%  [vdso]                                 [.] 0x0000000000000655
   0,81%  [nvidia]                               [k] _nv027733rm
   0,79%  [kernel]                               [k] clear_page_rep
   0,76%  [kernel]                               [k] psi_group_change
   0,70%  [kernel]                               [k] check_preemption_disabled
   0,69%  code                                   [.] 0x000000000623108f
   0,60%  code                                   [.] 0x0000000006231083
   0,59%  [kernel]                               [k] preempt_count_add
   0,54%  [kernel]                               [k] module_get_kallsym
   0,53%  [kernel]                               [k] copy_user_generic_string
```



Using a GUI to display perf data

- `perf report` is the default way to display perf data, directly in the console
- There are also graphical tools to display perf data:
  - Flamegraphs
    - Visualization based on hierarchical stacks
    - Allows to quickly find bottlenecks and explore the call stack
    - Popularized by Brendan Gregg tools which allows to generate flamegraphs from `perf` results.
  - Hotspot software
    - Developed and maintained by KDAB
    - A larger tool able to generate various types of visualizations from a `perf.data` file
    - Can also perform the actual perf recording



Visualizing data with flamegraphs

- Get the flamegraph scripts:

```
git clone https://github.com/brendangregg/FlameGraph fl
```

- Capture data:

```
perf record -g -- sleep 30
```

  - The `-g` option records call stacks for each sample

- Format the data:

```
perf script | ./fl/stackcollapse-perf.pl > out.perf-folded
```

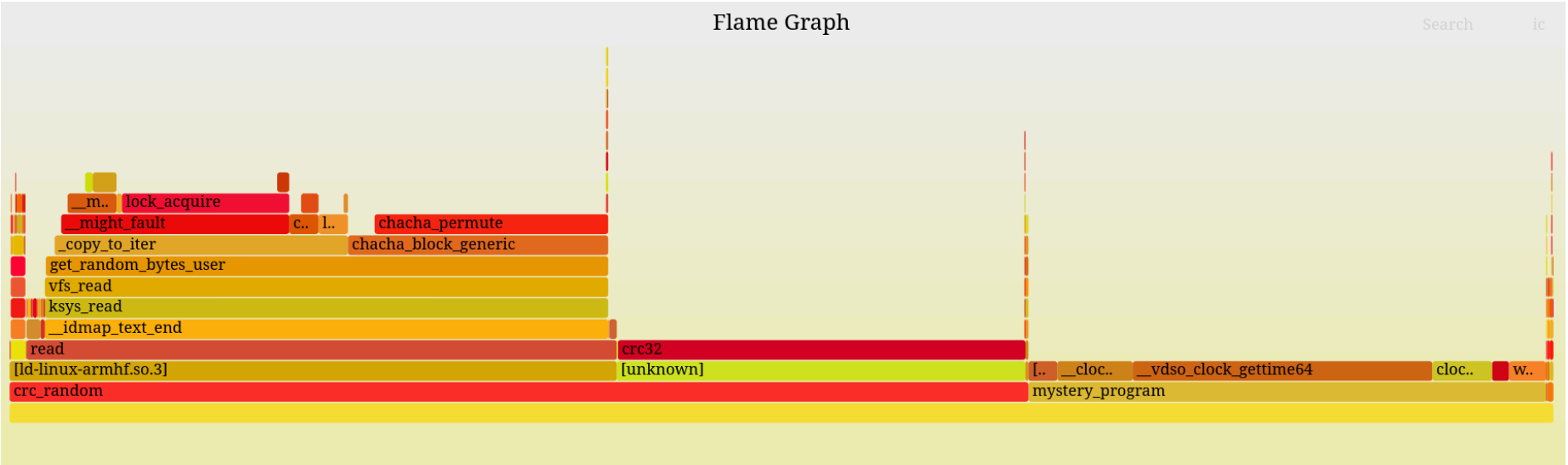
  - Other data sources are supported (eg: DTrace, SystemTap, Intel VTune, gdb...)

- Generate the Flamegraph:

```
./fl/flamegraph.pl out.perf-folded > flamegraph.svg
```

- The flamegraph can then be opened in a web browser

Flamegraph example: CPU flamegraph



- The plates on top represent the functions sampled by perf during the recording
- The plates width represents how often a function has been sampled by perf
- The plates below represent the call stacks for the sampled functions
- Flamegraphs are interactive: clicking on a plate will zoom on the corresponding callstack
- Colors can be tuned at flamegraph generation (eg: to get a clear split between kernel and userspace)



Visualizing data with hotspot (1/2)

- Designed to provide a frontend to perf data files
- Can generate flamegraphs on the fly, but not only:
  - CPU/tasks timelines
  - Interactive callstacks navigation
  - Code disassembly
- Configurable (eg: allows to set paths to find all needed debug information)

Visualizing data with hotspot (2/2)

